```
function vote(uint toProposal) public validPhase(Phase.Vote)  {        ❸

    Voter memory sender = voters[msg.sender];
    if (sender.voted || toProposal >= proposals.length) revert();      ❷
    sender.voted = true;
    sender.vote = toProposal;
    proposals[toProposal].voteCount += sender.weight;


}

function reqWinner() public validPhase(Phase.Done) view returns (uint ❸
➥ winningProposal) {                                                  ❹

    uint winningVoteCount = 0;
    for (uint prop = 0; prop < proposals.length; prop++)
        if (proposals[prop].voteCount > winningVoteCount) {
            winningVoteCount = proposals[prop].voteCount;
            winningProposal = prop;
        }

}
```

❶ Voting state change to be ordered by the chairperson

❷ Explicit validation using if statement

❸ validPhase modifier used in function headers

❹ View function, Tx not recorded on the chain

STORAGE VS. MEMORY VARIABLES

In the `vote` function, you'll find a local variable Voter `struct`. In Solidity, variables can be defined as `storage` (persistent and gets stored in the block) or `memory` (transient, does not get stored in the block). By default, simple variables are `memory` type, temporary, and do not get recorded in the block. The struct data structure is, by default, a `storage` variable, so you need to declare whether it is a memory or storage type when you use it. In the case of `vote` function, we've defined its local variable `Voter` struct as a `memory` type so that it does not waste storage in the block. When you define a struct inside a function as a local variable, you'll have to declare explicitly whether it is a memory or storage type.